Welcome to CSC331 Fall 2025 Data Structures

Professor Kevin Byron (kbyron@bmcc.cuny.edu)
Department of CIS, Room F-1030-I
Office hours: Mon 2pm-5pm (in-person and Zoom)
Office phone: 212-220-1490
Borough of Manhattan Community College/CUNY

CSC331 Fall 2025
Data Structures

Week 9:

Program 5 specifications
Recursion with linked list & tree
Binary search tree
AVL tree
Graph edge endpoint conversion

Professor Kevin Byron
Department of CIS
Borough of Manhattan Community College/CUNY

Reminders

- Program 3 due date
  - 1159pm Fri 10-31-2025
  - late submission receives no credit
- Exam 2
  - Section 501 Tue 11-11-2025 – Zoom 630pm-730pm
  - Section 172 Wed 11-12-2025 – In person room M-1001 630pm-730pm
    - Closed book, closed notes, no devices
  - Section 500 Wed 11-12-2025 – In person room N-453 1010am-1110am
    - Closed book, closed notes, no devices
- Program 4 due date
  - 1159pm Fri 11-21-2025
  - late submission receives no credit
- No class Thu 11-27-2025
- Program 5 due date
  - 1159pm Fri 12-12-2025
  - late submission receives no credit
- Posted on Brightspace
  - Week 9 lecture slides, C++ program with recursive linked list traversal (chap6b.cpp), C++ program to traverse binary search tree recursively (chap6d.cpp), sample C++ program to convert character graph edge endpoints to positional integers (chap12e.cpp), Practice quiz 2

Recursion

```
// chap6b.cpp
// kbyron@bmcc.cuny.edu
// 2-14-2019
// create linked list and display forward and backward w/recursion
// cloned from chap5a.cpp
#include <iostream>
using namespace std;
struct nodeType{
  int info;
  nodeType *link;
};
nodeType* buildListForward(){
  nodeType *first, *newNode, *last;
  int num;
  cout << "Enter a list of integers ending with -999." << endl;
  cin >> num;
  first = NULL;
  while (num != -999){
    newNode = new nodeType;
    newNode->info = num;
    newNode->link = NULL;
    if (first == NULL)
      first = newNode;
    else
      last->link = newNode;
    last = newNode:
    cin >> num;
  } //end while
return first;
} //end buildListForward
```

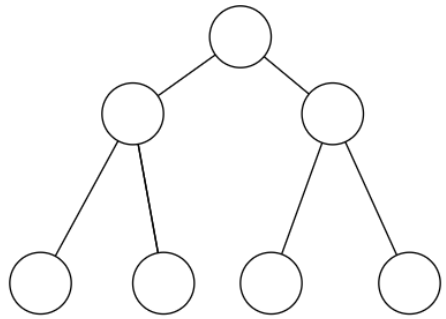
Recursion

```
void forward(nodeType *p){
  if(p!=NULL){
    cout << p->info << " ";
    forward(p->link);
  }
}
void backward(nodeType *p){
  if(p!=NULL){
    backward(p->link);
    cout << p->info << " ";
  }
}
int main(){
  nodeType *p,*head;
  head=buildListForward();
  cout << "original list: ";
  forward(head);
  cout << endl;
  cout << "backward list: ";
  backward(head);
  cout << endl;
}
```

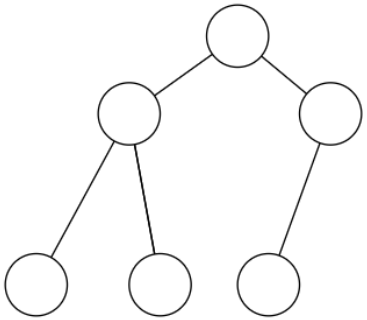
```
C:\Users\Kevin Byron\work\cpp\fy19>chap6b < chap6d.dat
Enter a list of integers ending with -999.
original list: 22 33 11 44 22 3 2 1
backward list: 1 2 3 22 44 11 33 22
```

Recursion with trees

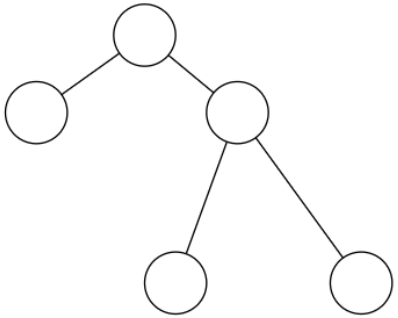
- A binary tree has no more than two child nodes for any node
- Categories of binary trees include:
  - Full binary tree: every non-leaf node has exactly two child nodes
  - Complete binary tree: every level, except possibly the last, is completely filled, and all nodes are as far left as possible
  - Perfect binary tree: every leaf node is at the same level and every internal node has two child nodes
- Examples:



Complete, full and perfect binary tree.



Complete binary tree.

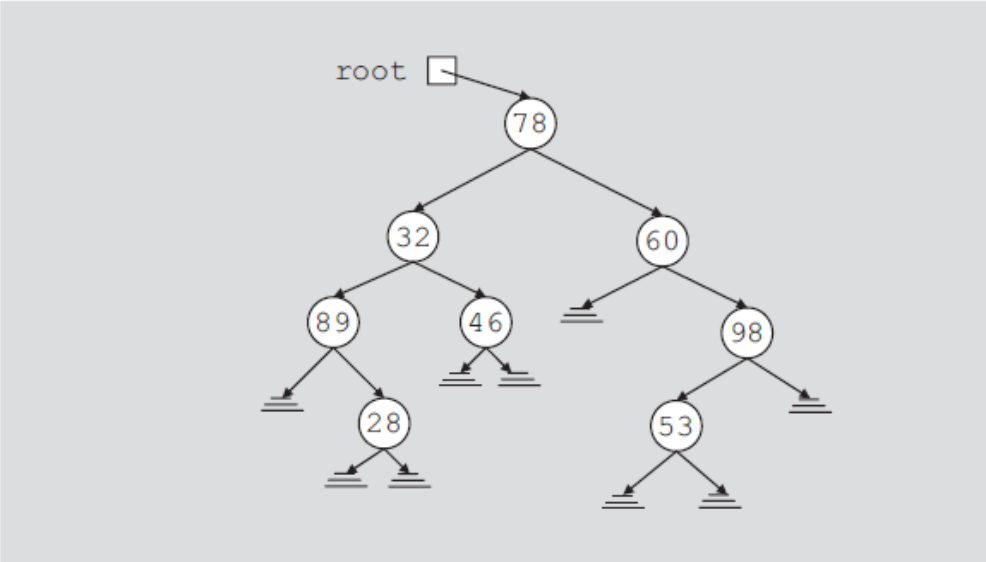


Full binary tree.

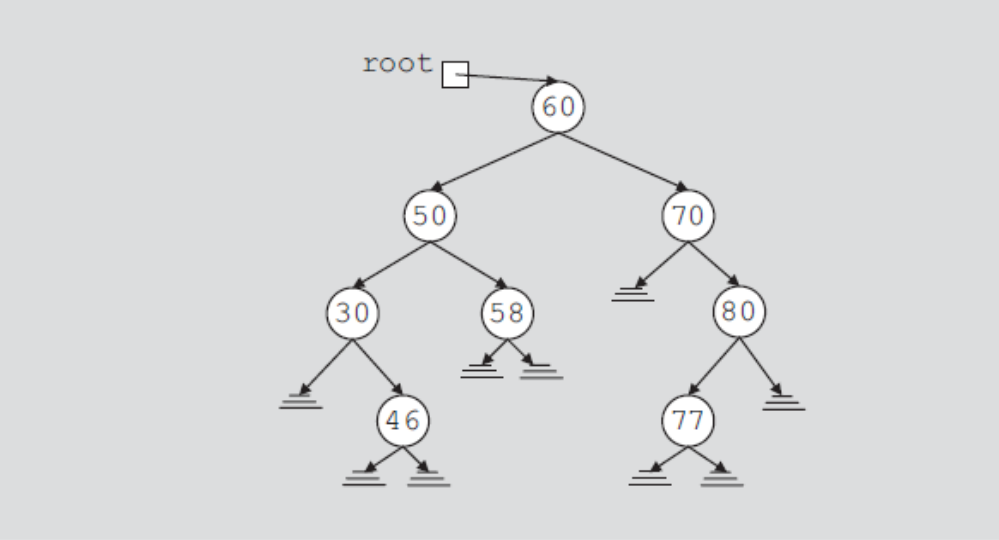
Recursion on a binary search tree

- A binary search tree is a binary tree with the following property:
  - For each node in the tree, any descendant on the left side has a value less than the value of this node and any descendent on the right side has a value greater that the value of this node
- A simple binary search tree is created as follows:
  - A new node is added relative to preexisting nodes, if any
  - The position of a pre-existing node does not change
  - A new node is added as a leaf node, i.e., the new node has no child nodes
  - A node with a value that is already in the tree will not be added; each node in the tree is unique

Recursion on a binary search tree



Arbitrary binary tree

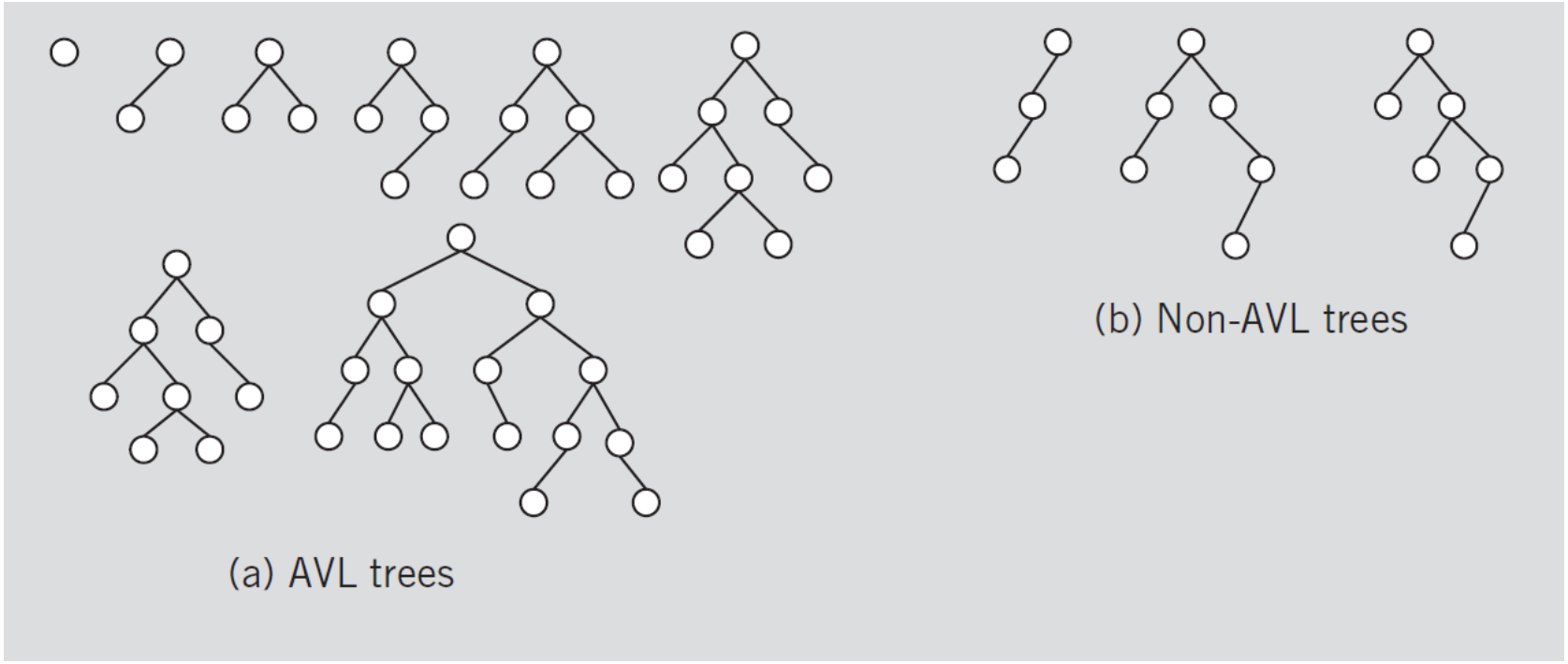


Binary search tree

Recursion with trees

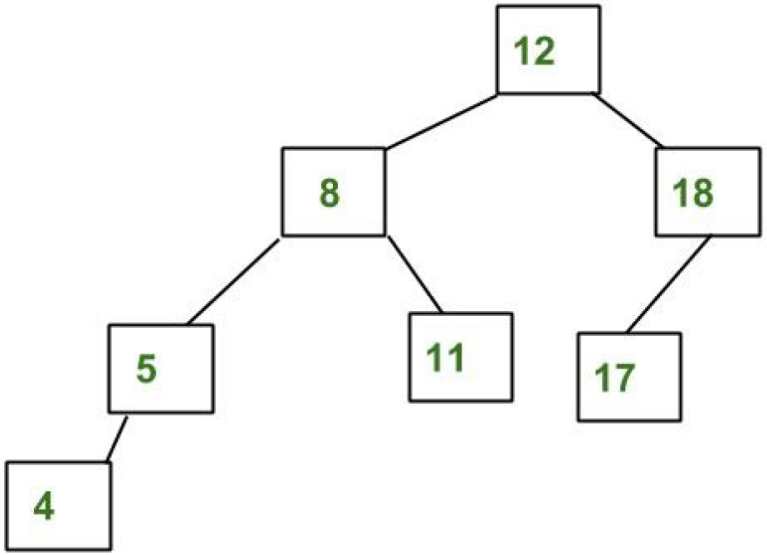
- The height of a tree is measured in edges to the most distant leaf node
- A binary tree is considered balanced if the heights of the right and left subtrees of each node differ by at most one edge
- A binary search tree (BST), sometimes called an ordered or sorted binary tree, is a data structure that allows O(log n) speed operations (i.e., lookup, addition and deletion) when the BST is balanced
- In an unbalanced BST, an operation may require up to O(n) time
- An AVL tree (named after inventors Adelson-Velsky and Landis) is a self-balancing BST
- Creating and traversing an AVL is an efficient method to sort items

Recursion with trees

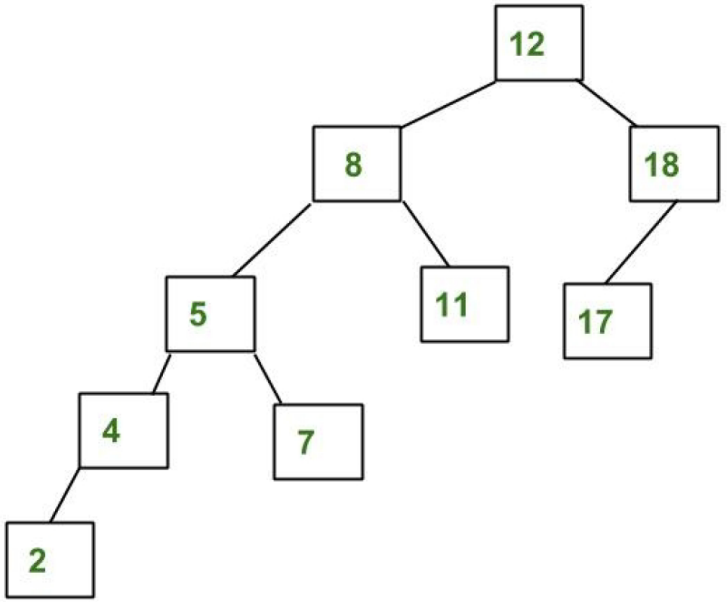


Recursion with trees

An Example Tree that is an AVL Tree



An Example Tree that is NOT an AVL Tree

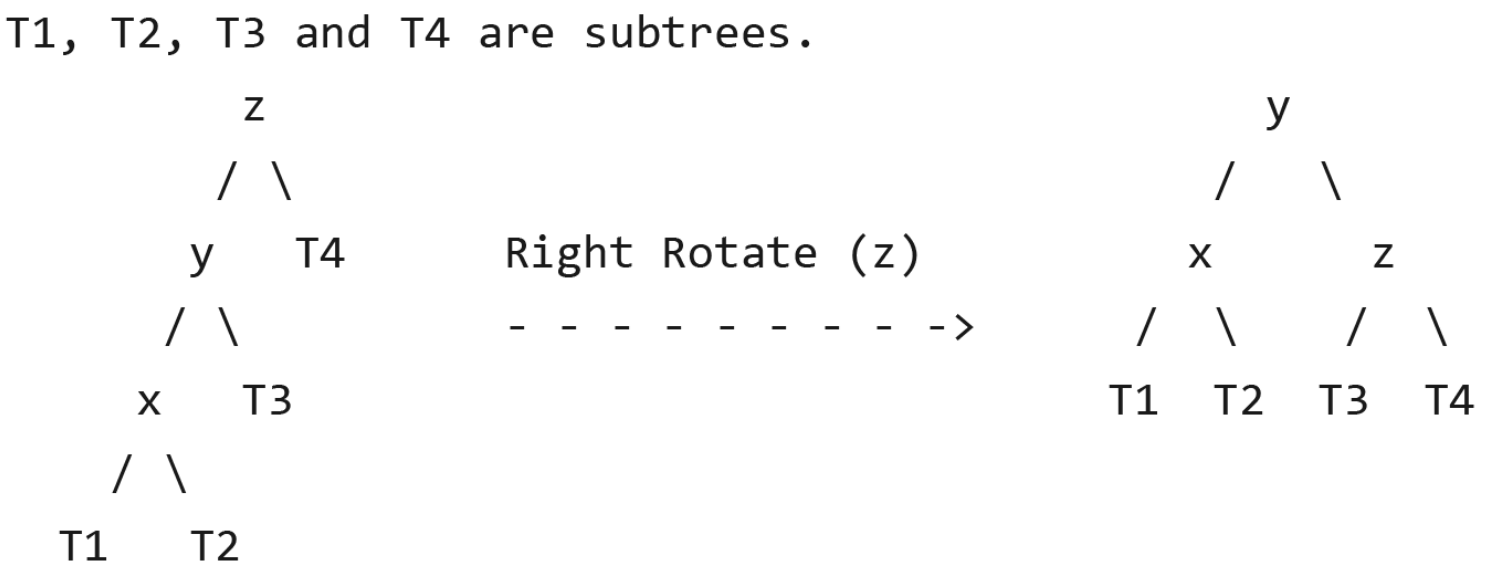


Recursion with trees

- Each node of an AVL tree has a height and balance factor (BF)
- The height of a node is 1 + max(left subtree height, right subtree height)
- The BF is the difference of right subtree height minus left subtree height
- When BF < -1 or > 1, one of 4 adjustments, known as rotations, is performed to maintain a balanced tree

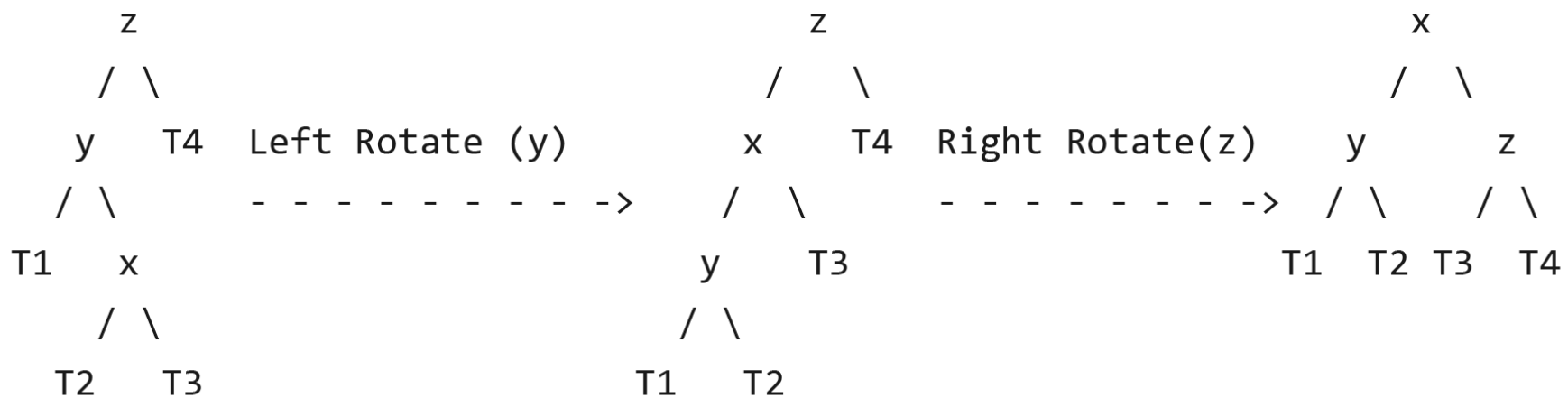
Recursion with trees

a) Left Left Case



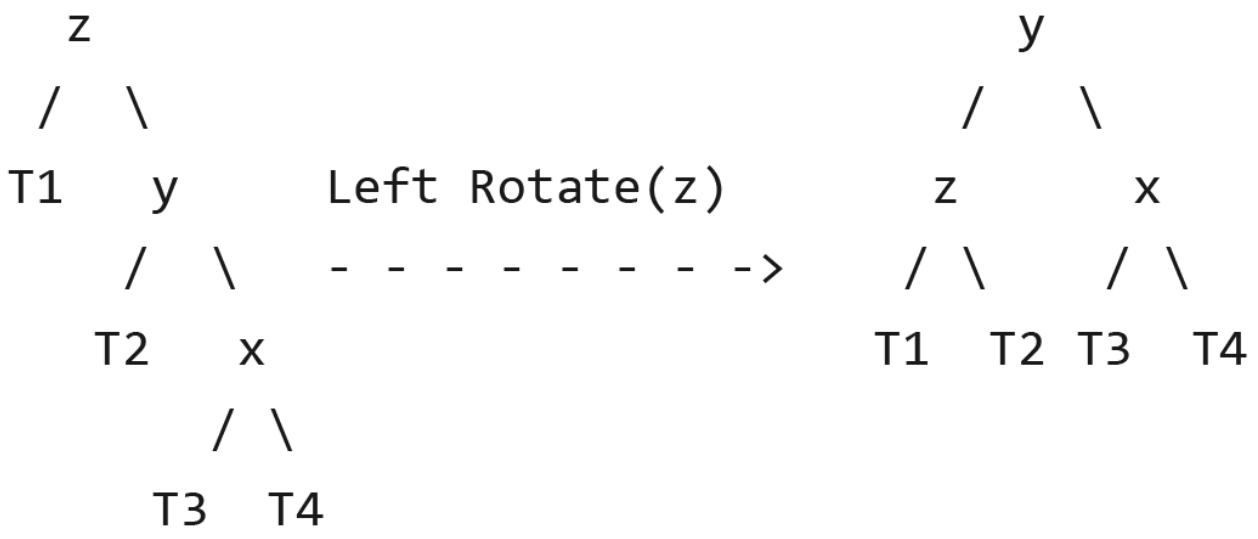
Recursion with trees

b) Left Right Case



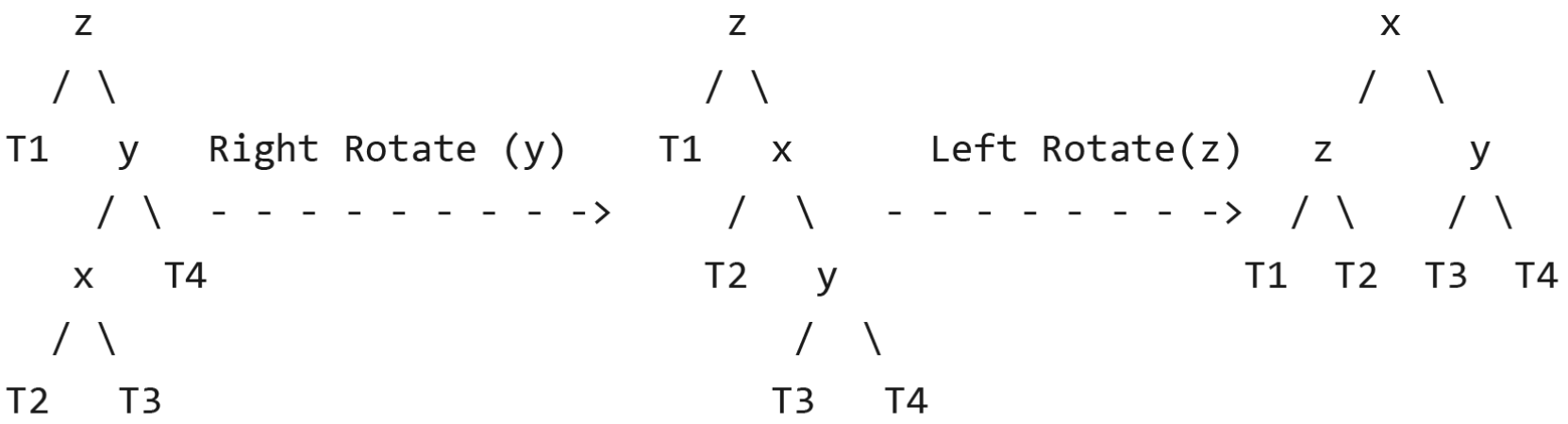
Recursion with trees

c) Right Right Case

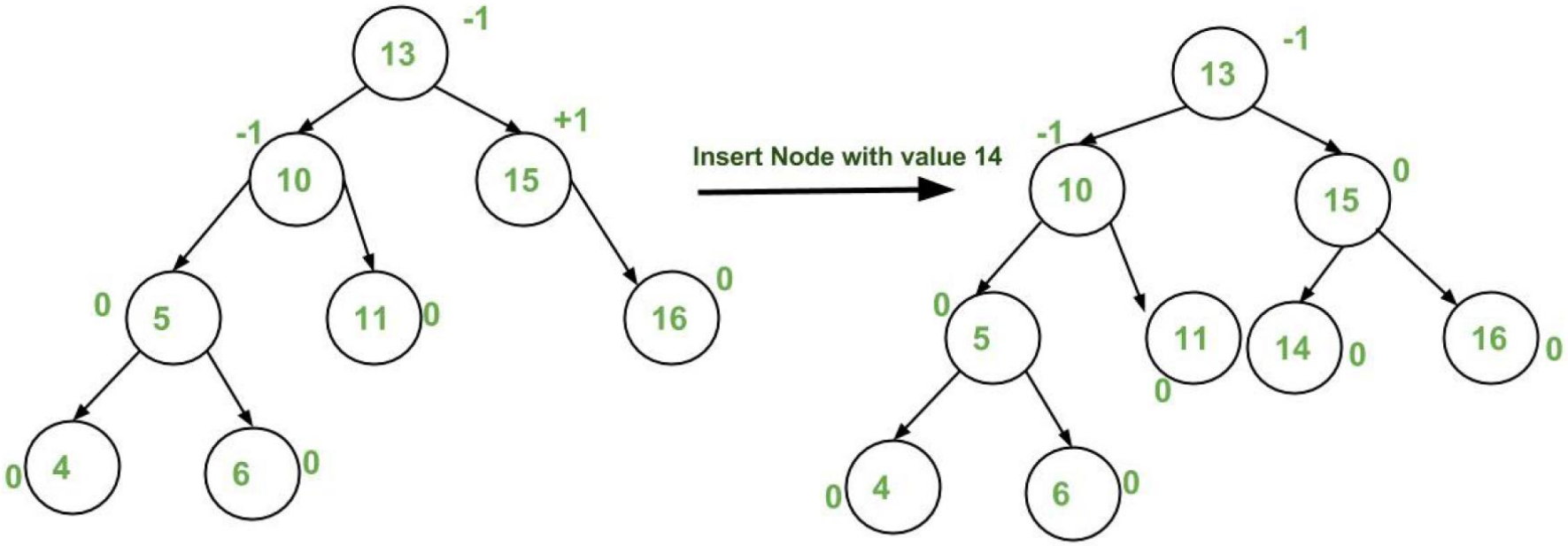


Recursion with trees

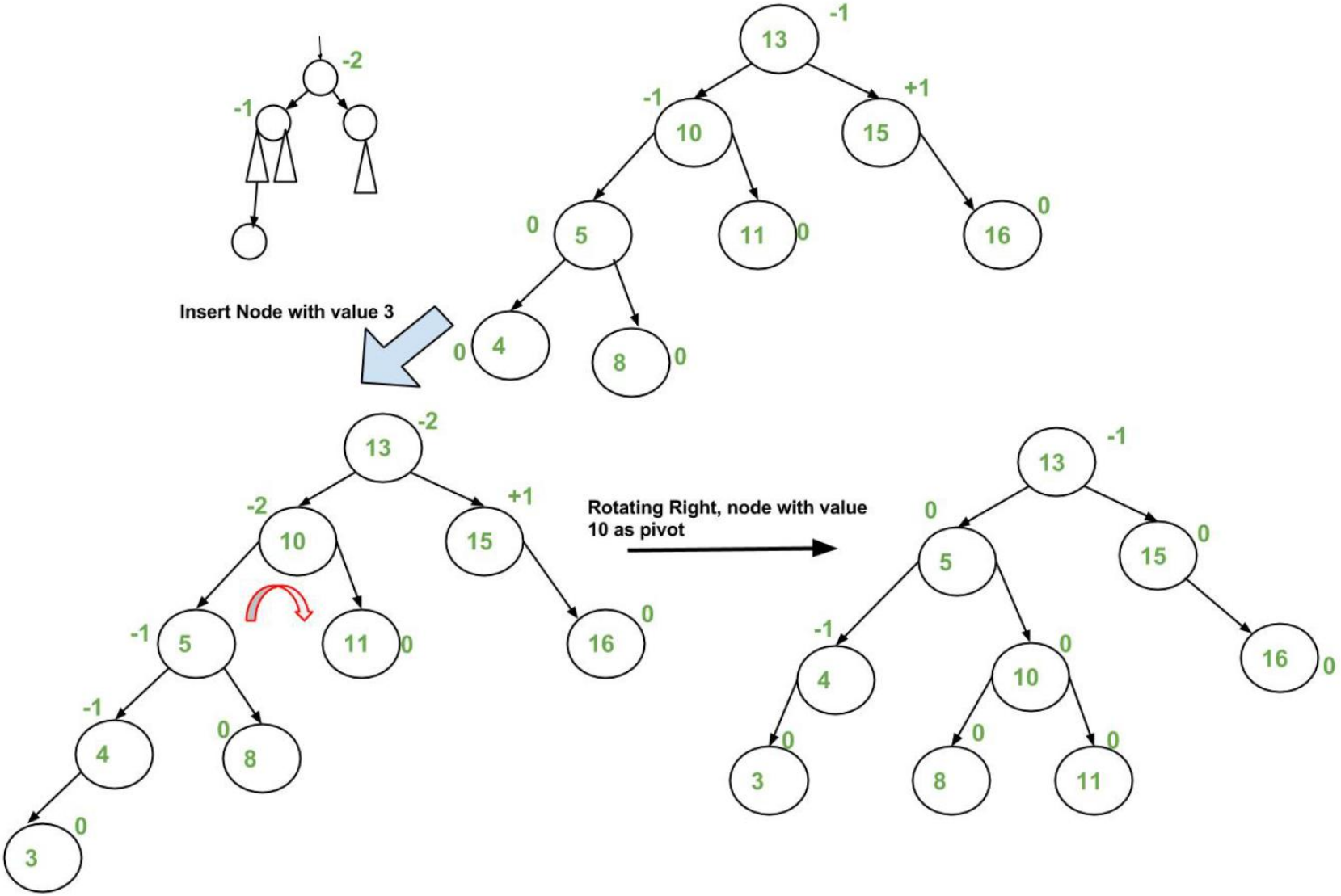
d) Right Left Case



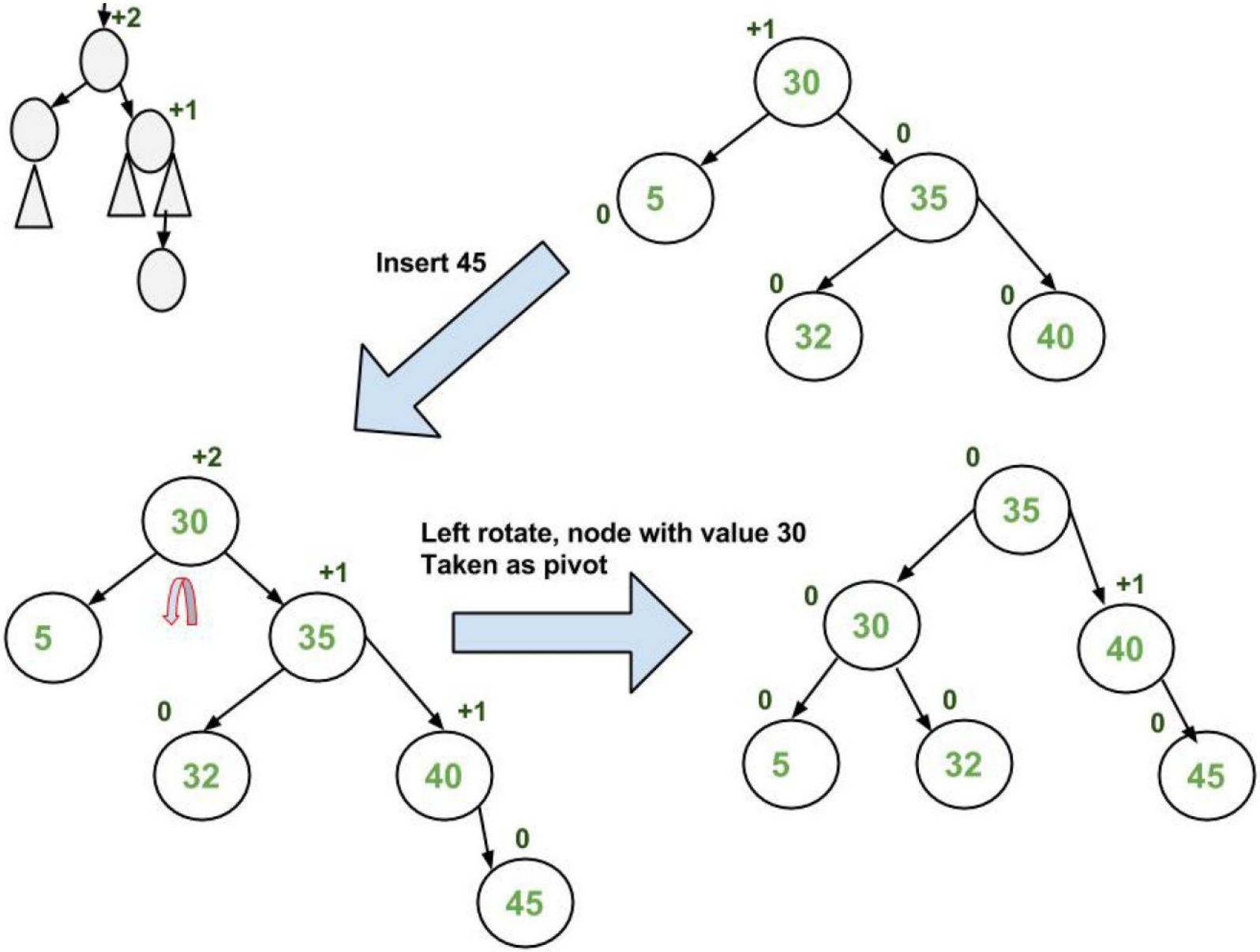
Recursion with trees



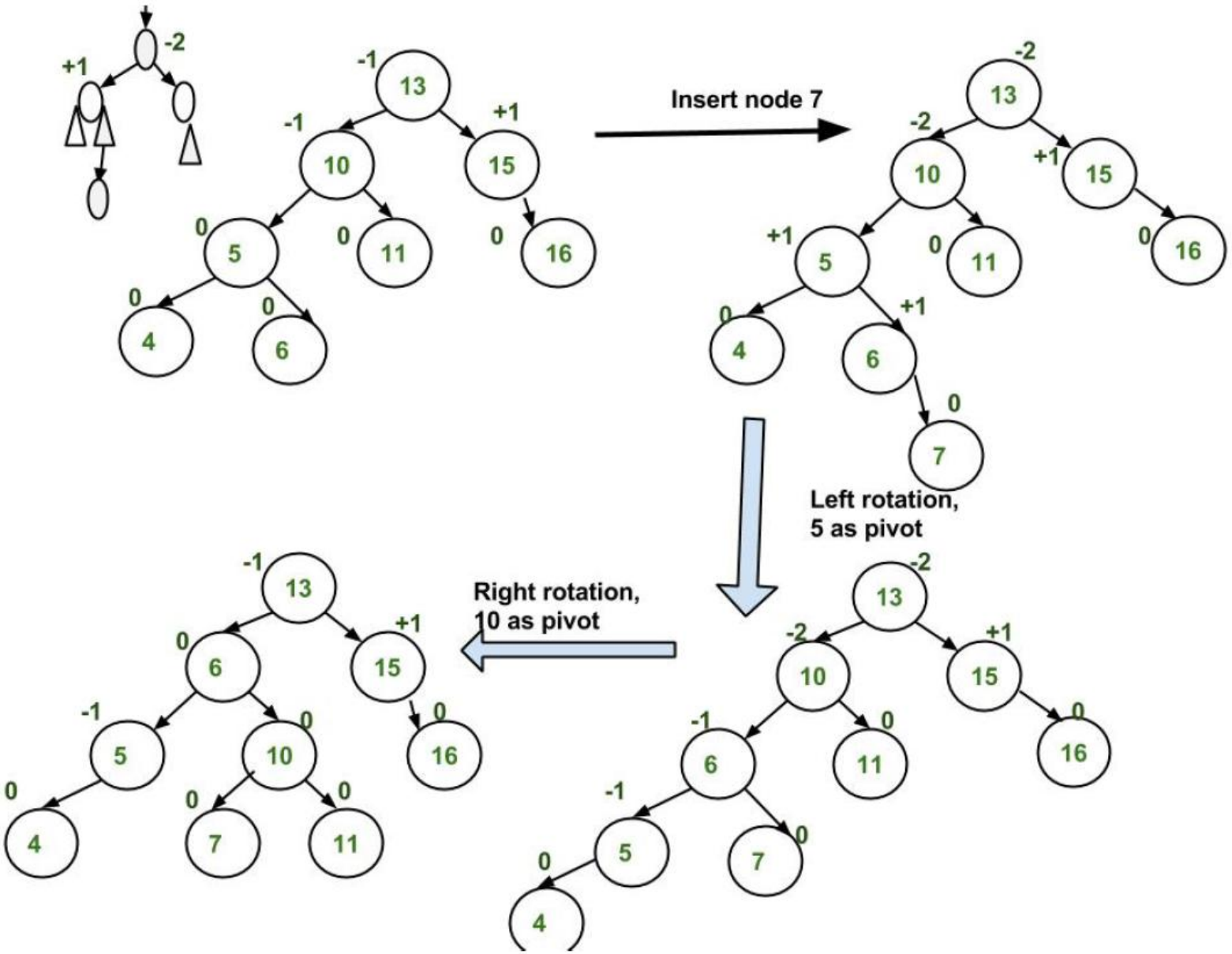
Recursion with trees



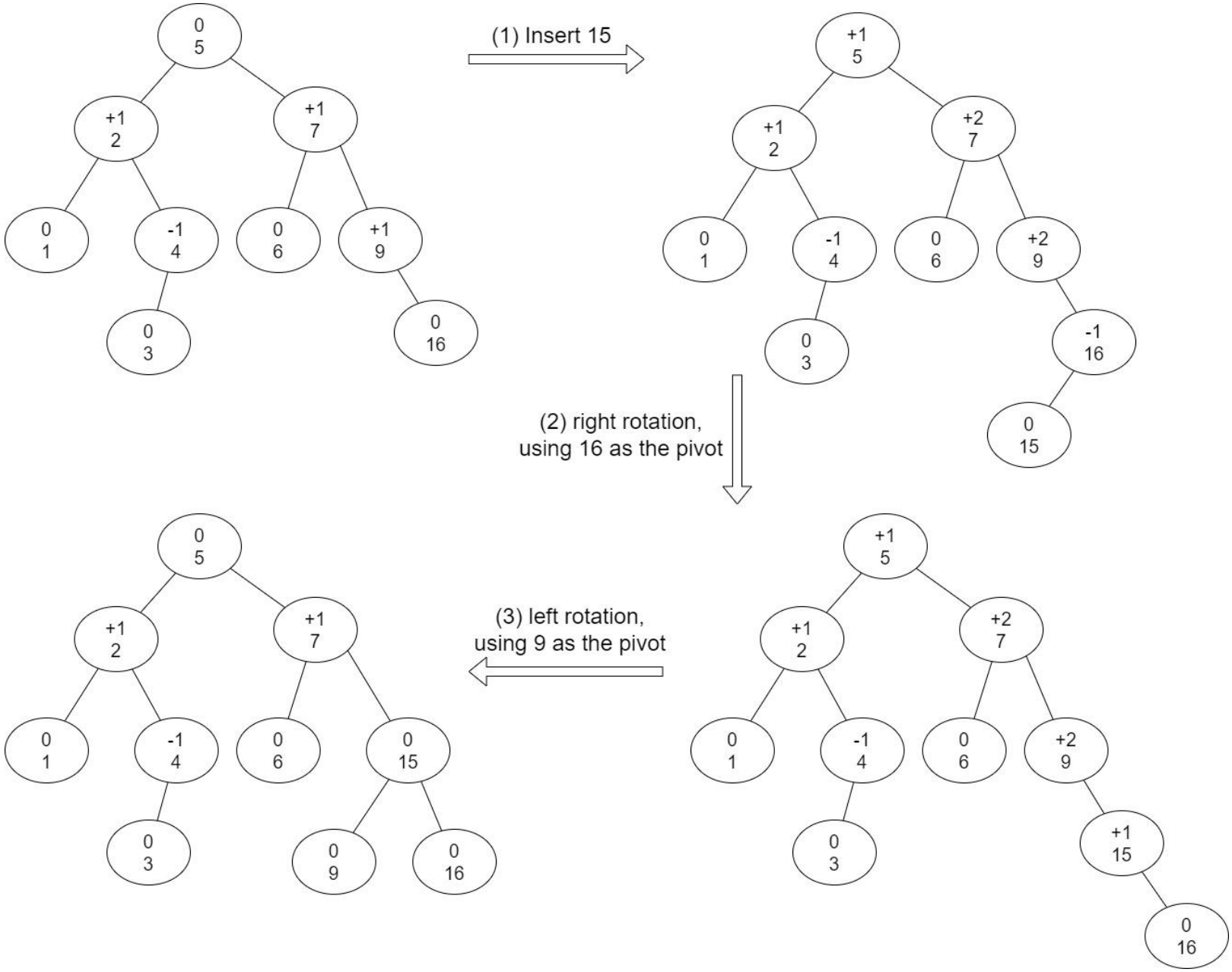
Recursion with trees



Recursion with trees



Recursion with trees



Recursion with trees

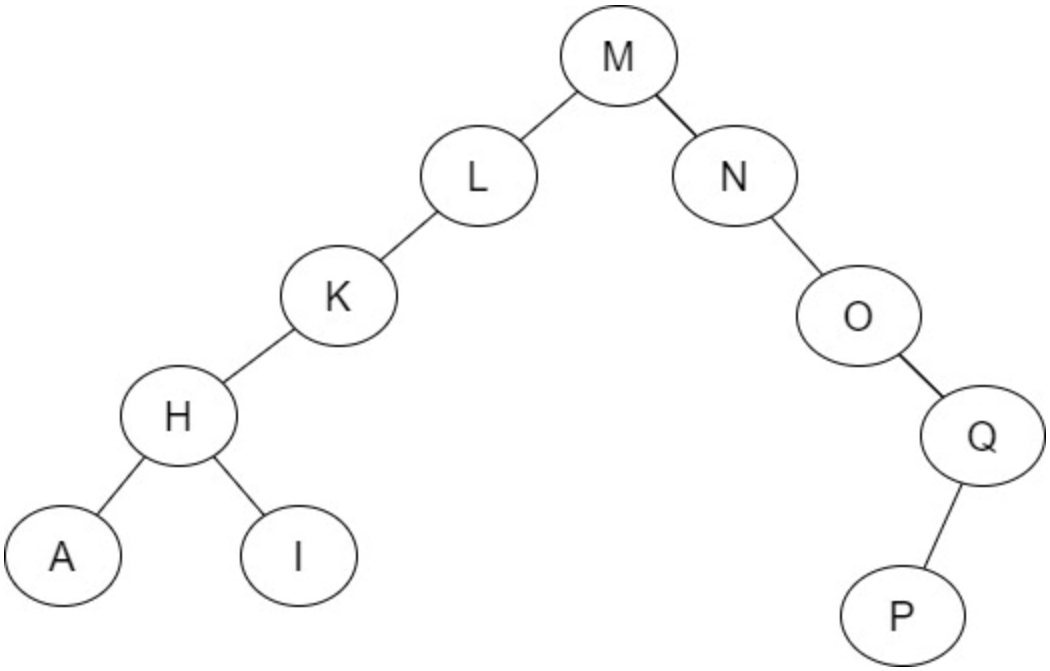
- Example: create
  - (1) a BST without balancing (no swapping) using these 10 non-numeric keys:

    M, N, O, L, K, Q, P, H, I, A
  - (2) an AVL tree using these 10 non-numeric keys: M, N, O, L, K, Q, P, H, I, A
  - (3) an AVL tree using these 10 non-numeric keys: A, I, H, P, Q, K, L, O, N, M

Recursion with trees

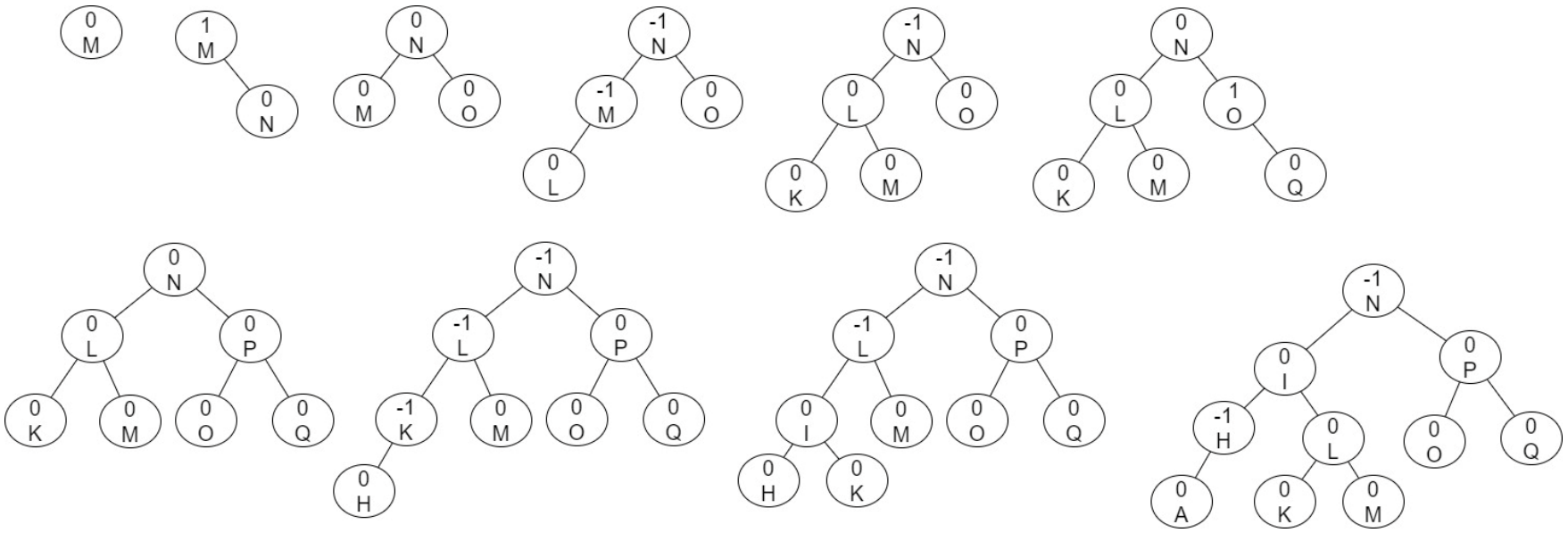
- Solution: create
  - (1) a BST without balancing (no rotating) using these 10 non-numeric keys:

    M, N, O, L, K, Q, P, H, I, A



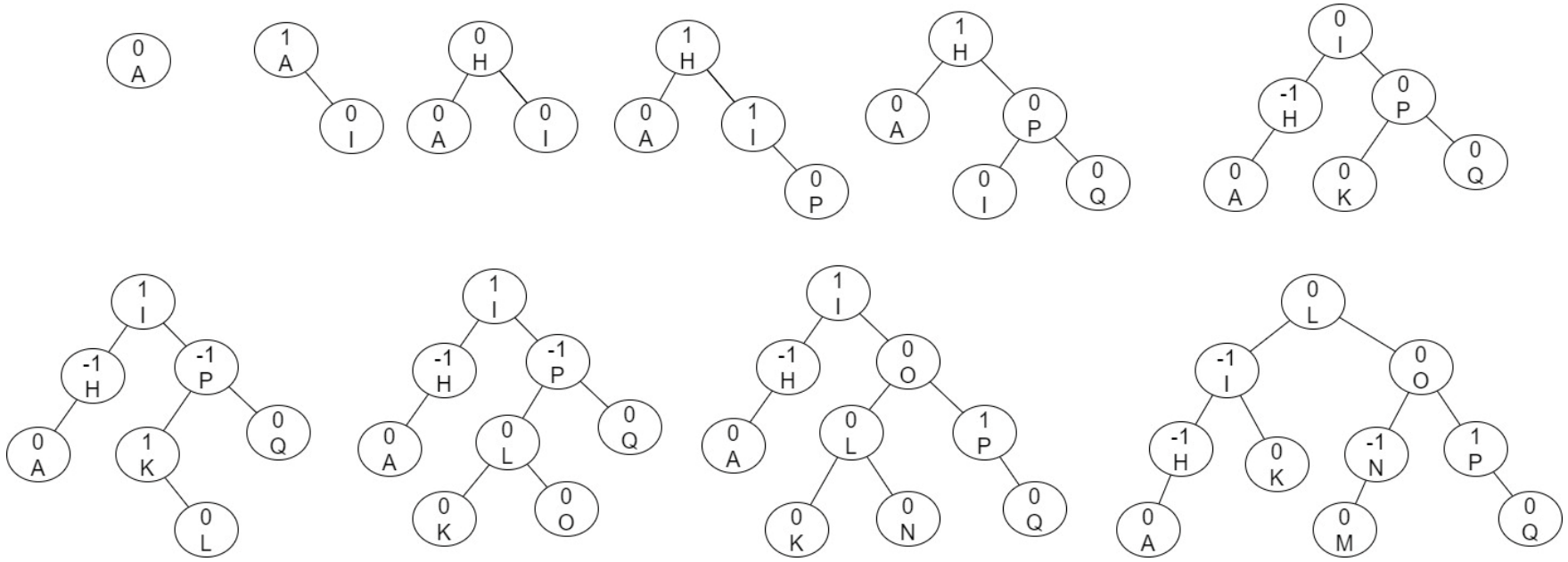
Recursion with trees

- Solution: create
  - (2) an AVL tree using these 10 non-numeric keys: M, N, O, L, K, Q, P, H, I, A



Recursion with trees

- Solution: create
  - (3) an AVL tree using these 10 non-numeric keys: A, I, H, P, Q, K, L, O, N, M



Recursion with trees

```
// chap6d.cpp
// kbyron@bmcc.cuny.edu
// 12-5-2022
// create binary search tree and traverse w/recursion
// cloned from fy19/chap6d.cpp
#include <iostream>
using namespace std;
struct nodeType{
  int info;
  nodeType *lLink;
  nodeType *rLink;
};
//-------------------------------------------------------
void treeAdd(nodeType* &root, int num){
  if (root == NULL){
    root = new nodeType;
    root->info = num;
    root->lLink = NULL;
    root->rLink = NULL;
  }
  else
    if (num==root->info)
      cout << "duplicate not added: " << num << endl;
    else{
      if (num<root->info)
        treeAdd(root->lLink,num);
      else
        treeAdd(root->rLink,num);
    }
}
//-------------------------------------------------------
```

Recursion with trees

```
//-------------------------------------------------
nodeType* buildTree(){
  int num;
  nodeType *root;
  cout << "Enter a list of integers ending with -999." << endl;
  cin >> num;
  root = NULL;
  while (num != -999){
    treeAdd(root,num);
    cin >> num;
  }
  return root;
}
//-------------------------------------------------
void showTree(nodeType *p){
  // cout << "in showTree ... " << endl;
  if(p!=NULL){
    showTree(p->lLink);
    cout << p->info << " ";
    showTree(p->rLink);
  }
}
//-------------------------------------
```

```
int main(){
  nodeType *root;
  root=buildTree();
  cout << "tree contents: ";
  showTree(root);
  cout << endl;
}
```

Recursion with trees

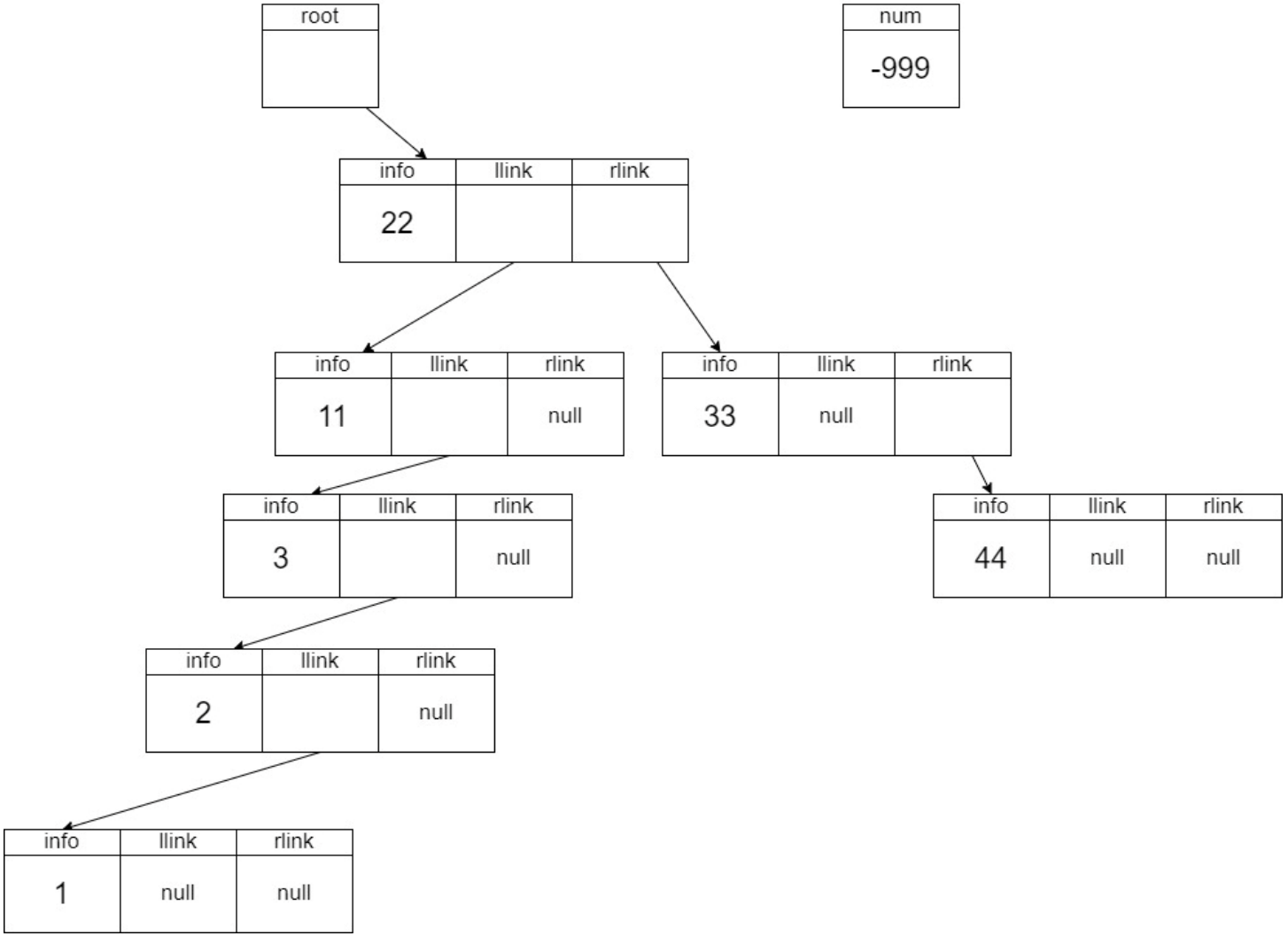
- Execution of chap6d using redirection …

```
C:\Users\Kevin Byron\work\cpp\fy19>type chap6d.dat
22
33
11
44
22
3
2
1
-999

C:\Users\Kevin Byron\work\cpp\fy19>chap6d < chap6d.dat
Enter a list of integers ending with -999.
tree contents: 1 2 3 11 22 33 44
```

Recursion with trees

- Final BST after Execution of chap6d …



Recursion with trees

- Practice problems using chap6d.cpp
  - Change showtree function to perform a pre-order tree traversal
  - Show output produced when showtree performs a pre-order tree traversal
  - Change showtree function to perform a post-order tree traversal
  - Show output produced when showtree performs a post-order tree traversal

Edge endpoint conversion

– Sample input edges: graph1.dat:

```
A X 1
A M 6
A W 2
M Q 7
M W 5
Q X 4
W X 3
end-of-file
```

Edge endpoint conversion

– Conversion: ./chap12e graph1.dat

```
Graph vertices are:  A, M, Q, W, X.
Graph size is 5.
vertices = AMQWX
old edge = A X 1 ... new edge = 0 4
old edge = A M 6 ... new edge = 0 1
old edge = A W 2 ... new edge = 0 3
old edge = M Q 7 ... new edge = 1 2
old edge = M W 5 ... new edge = 1 3
old edge = Q X 4 ... new edge = 2 4
old edge = W X 3 ... new edge = 3 4
```

Assignment

- Read Malik chapter 11 – AVL trees
- https://www.geeksforgeeks.org/avl-tree-set-1-insertion/